

Relatório 9 - Prática: Git e Github para Iniciantes (III)

João Pedro Gomes

1. Introdução

O card 9 pede pra assistirmos uma série de 25 videos sendo 24 na pasta do LAMIA e 1 no youtube sobre git e github e depois escrever esse relatório explicando o conhecimento adquirido nos vídeos.

2. Desenvolvimento

O git ajuda no controle de versão de projetos, controle de versão ajuda em organizar todas as versões que um projeto pode ter ele salva todas as mudanças feitas no projeto e salva como uma versão, o git funciona de uma forma diferente das outras plataformas de versionamento enquanto elas comparam o estado de cada arquivo pra salvar a nova versão e só salvando o que mudou o git tira snapshots e ele salva a partir do estado do arquivo.



O que tentei mostrar aqui mesmo sabendo dos estados de um arquivo e quando o git classifica um arquivo como modified ele ainda não foi commitado e depois de commitado ele volta pra unmodified, aqui eu só quero dar um exemplo de que na versão 1 ele salvou os arquivos do projeto, na versão 2 salvou o estado dos 2 primeiros arquivos que foram modificados e na versão 3 salvou o estado novamente dos 2 arquivos editados e mais um arquivo adicionado, separando cada modificação em uma versão.

Como falei dos estados dos arquivos agora vou explicar detalhadamente como eles funcionam, quando você está em um projeto usando o git seus arquivos

passam por 4 status, untracked, unmodified, modified e staged, e eles significam isso:

- Untracked: Um arquivo novo que o git ainda não reconhece e só irá passar a reconhecer depois de adicionarmos ele no próximo commit.
- Unmodified: Um arquivo que o git já reconhece mas que ainda não mudou desde o ultimo commit.
- Modified: Um arquivo que foi modificado desde o ultimo commit.
- Staged: Quando o arquivo já está adicionado e pronto pra ser commitado, e depois de commitado o arquivo se modificado anteriormente volta pro estado de unmodified.

Durante as aulas ele vai ensinando alguns códigos que irei explicar agora:

- git init: cria um repositório Git na pasta atual
- git status: mostra o estado dos arquivos (untracked, modified, staged)
- git add (arquivo): envia o arquivo para a área de stage
- git add . : envia todas as alterações para o stage pra serem commitadas
- git commit -m "mensagem": manda as alterações feitas pro github com uma mensagem escrita
- git log: mostra o histórico de commits
- git log --oneline: mostra o histórico resumido em uma linha por commit
- git diff: mostra as mudanças feitas que ainda não foram para o stage
- git diff --staged: mostra as mudanças que já estão no stage
- git restore arquivo.txt: desfaz alterações no arquivo (volta ao último commit)
- git restore --staged arquivo.txt: remove o arquivo do stage
- git rm arquivo.txt: remove o arquivo do projeto e do Git

Depois ele fala um pouco sobre os repositórios remotos que é o github, ele é um site que gerencia os projetos git e guarda eles na nuvem e permite que qualquer

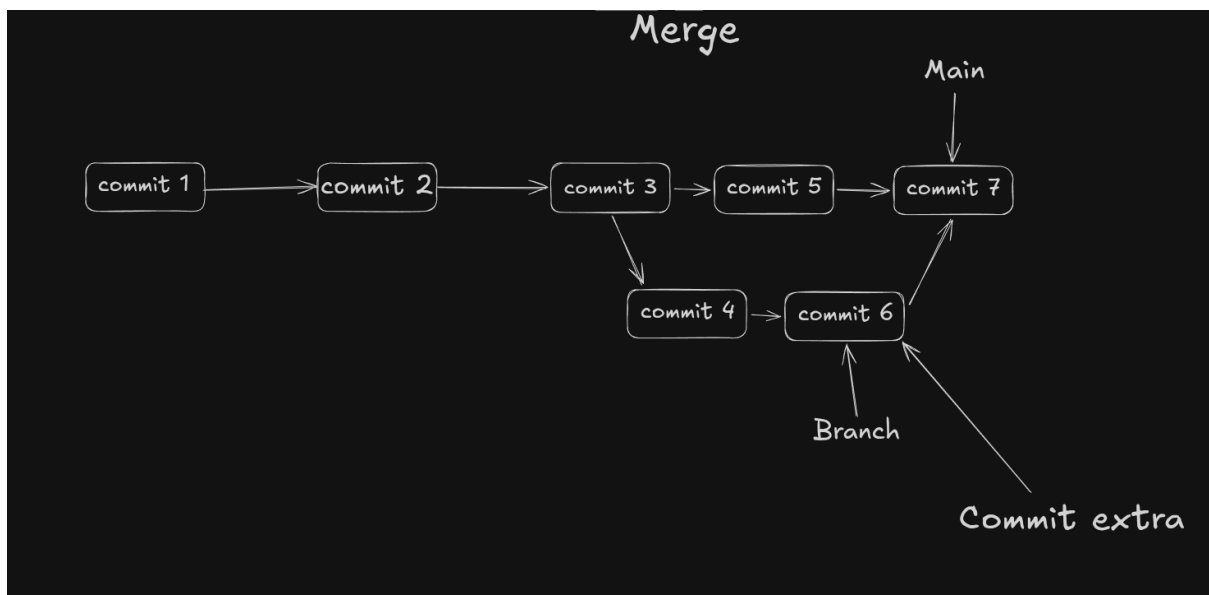
peessoa veja o projeto e colabore fazendo um Fork que é fazer uma copia do projeto para outro repositório pra ela fazer as alterações e depois fazer um pedido para mandar as alterações que ela fez no seu projeto e também qualquer pessoa pode clonar seu projeto e baixar ele pra utilizar mas se ela fizer alterações ela não pode fazer o pedido pra enviar pro seu projeto pois ela só clonou e não fez um Fork.

E existem códigos pra trabalhar com o github também, e são eles:

- `git clone (URL)`: clona um projeto pra pasta onde você está
- `git remote add origin (URL)`: conecta uma pasta local onde está o projeto com um repositório do github
- `git remote -v`: mostra os repositórios remotos que estão conectados
- `git push -u origin main`: envia o código pro github pela primeira vez e define a branch padrão
- `git push`: envia os arquivos commitados pro github
- `git pull`: puxa as mudanças feitas pro projeto local
- `git branch -r`: lista as branches que existem

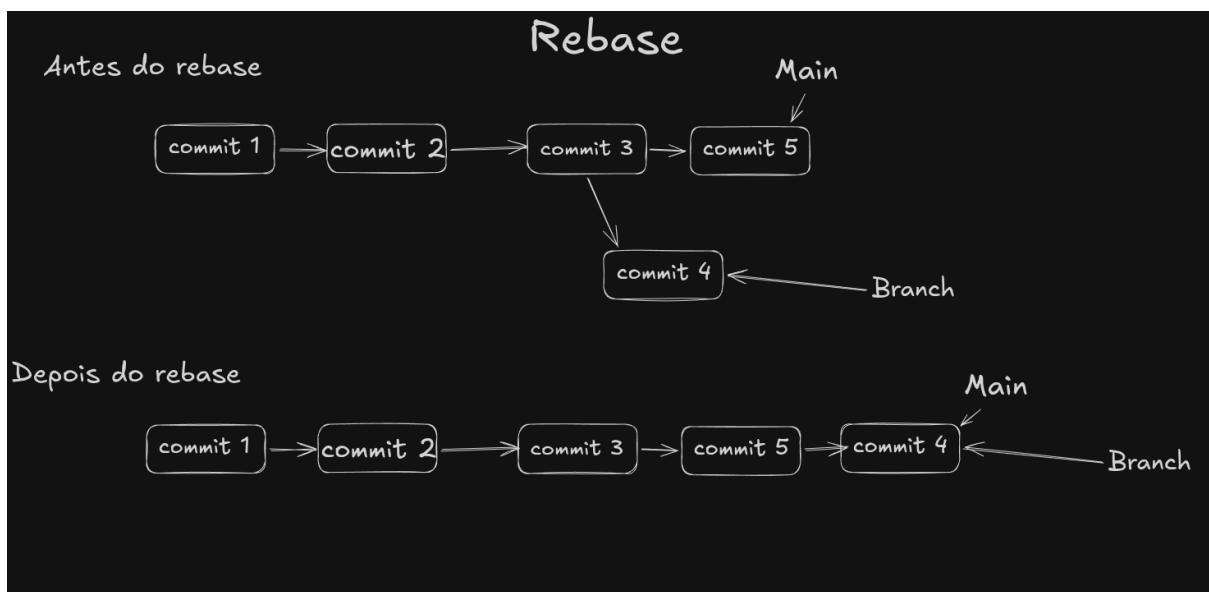
E também existem as branches, uma branch é um ponteiro que aponta pra algum commit e é usado pra dividir tarefas e não causar conflitos em uma equipe, por exemplo, se alguém precisa resolver um bug específico ele cria um branch resolve e pode fazer testes sem atrapalhar mais ninguém pois ele está fazendo isso separadamente em uma branch. E depois de finalizar o trabalho em uma branch e commitar as mudanças tem que ser feito o pull request, um pull request é um pedido pra unir o código alterado com a main depois de uma revisão e existem 2 formas pra unir com o merge e o rebase.

Merge:



O merge junta o histórico da branch com a main aí quando comita ele junta essas linhas fazendo um commit extra juntando elas, se tiver muitas branches essa grande quantidade de commit pode poluir o visual no histórico.

Rebase:



Diferente do merge onde tem commits extras e a linha do histórico toda poluída, com o rebase ele já deixa a linha do historico totalmente reta e sem poluição, oq ele faz é pegar o commit do branch e jogar pra frente do ultimo commit feito na main, deixando ele linear.

Pra trabalhar com o github existe um app pra download que se chama GitHub Desktop que facilita pra iniciante que não querem ou não sabem mexer com comandos no terminal, ele tem uma interface grafica intuitiva que permite clonar projetos, ver mudanças que algum commit esta fazendo, ver historicos de commits, ver as as branches que existem etc.

3. Conclusão

Aprender sobre git e github é muito importante pra qualquer pessoa que trabalha na area da computação, esse card é muito importante pra adquirir conhecimentos que eu ainda não tinha sobre o git e aperfeiçoar os que eu já tinha, os videos foram fáceis de entender e com uma explicação muito didática e de forma pratica.